

Курсовая работа

Задача 1.1 — ETL-загрузка банковских CSV в детальный слой DS PostgreSQL

Студент: Орисенко Максим

Дата: 08.05.2026

1. Постановка задачи

По легенде, банк выполнил миграцию ядровой БД, в момент переключения часть данных за конец 2017 — начало 2018 года осталась в старой системе и была выгружена в шесть CSV-файлов. Старая БД отключена, для построения 101-й формы эти данные необходимо загрузить в новый детальный слой DS централизованного хранилища на PostgreSQL.

По требованию задания нужно реализовать следующее:

- создать схему DS и шесть таблиц под исходные файлы;
- создать отдельную схему LOGS и логовую таблицу со временем старта/окончания и доп. статистикой;
- после логирования START добавить таймер на 5 секунд, чтобы START и END были визуально разнесены во времени;
- реализовать обновление по первичным ключам в режиме «Запись или замена», для FT_POSTING_F — полная перезагрузка;
- учесть разные форматы дат в разных файлах.

2. стек и архитектура решения

PostgreSQL 13 поднят в Docker (контейнер из соседнего проекта hw_airflow_task7), на хосте порт 5433 (порт 5432 был занят локальным сервисом PostgreSQL). База airflow, пользователь airflow, пароль airflow. ETL-процесс реализован на Python 3 с библиотекой psycopg2 — это удовлетворяет технологическим требованиям (допускаются Python / Java / Scala / Talend). Вторая, альтернативная, реализация — поток на Talend Open Studio for DI 7.1.1.

Поднятый контейнер с Postgres (docker ps) виден на скриншоте — порт 5433 проброшен на хост, статус healthy:

```
Командная строка
Microsoft Windows [Version 10.0.22631.6199]
(c) Microsoft Corporation. All rights reserved.

C:\Users\maksim>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
666b6b7c19ea   postgres:13    "docker-entrypoint.s..." 19 hours ago   Up 55 minutes (healthy)   0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
1942e2570f65   apache/airflow:2.8.1 "/usr/bin/dumb-init ..." 19 hours ago   Up 55 minutes (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
50b2f87d31b7   apache/airflow:2.8.1 "/usr/bin/dumb-init ..." 19 hours ago   Up 55 minutes (healthy)   8080/tcp
edfd127b0105   apache/airflow:2.8.1 "/usr/bin/dumb-init ..." 19 hours ago   Up 55 minutes (healthy)   8080/tcp
b09116ea9758   apache/airflow:2.8.1 "/usr/bin/dumb-init ..." 19 hours ago   Up 55 minutes (healthy)   8080/tcp
041342a84455   redis:latest    "docker-entrypoint.s..." 19 hours ago   Up 55 minutes (healthy)   6379/tcp
hw_airflow_task7-postgres-1
hw_airflow_task7-airflow-webserver-1
hw_airflow_task7-airflow-worker-1
hw_airflow_task7-airflow-triggerer-1
hw_airflow_task7-airflow-scheduler-1
hw_airflow_task7-redis-1

C:\Users\maksim>
```

Рис. 1. Postgres 13 в контейнере hw_airflow_task7-postgres-1 на порту 5433.

DBeaver успешно подключается к этой БД (драйвер PostgreSQL JDBC 42.7.2):

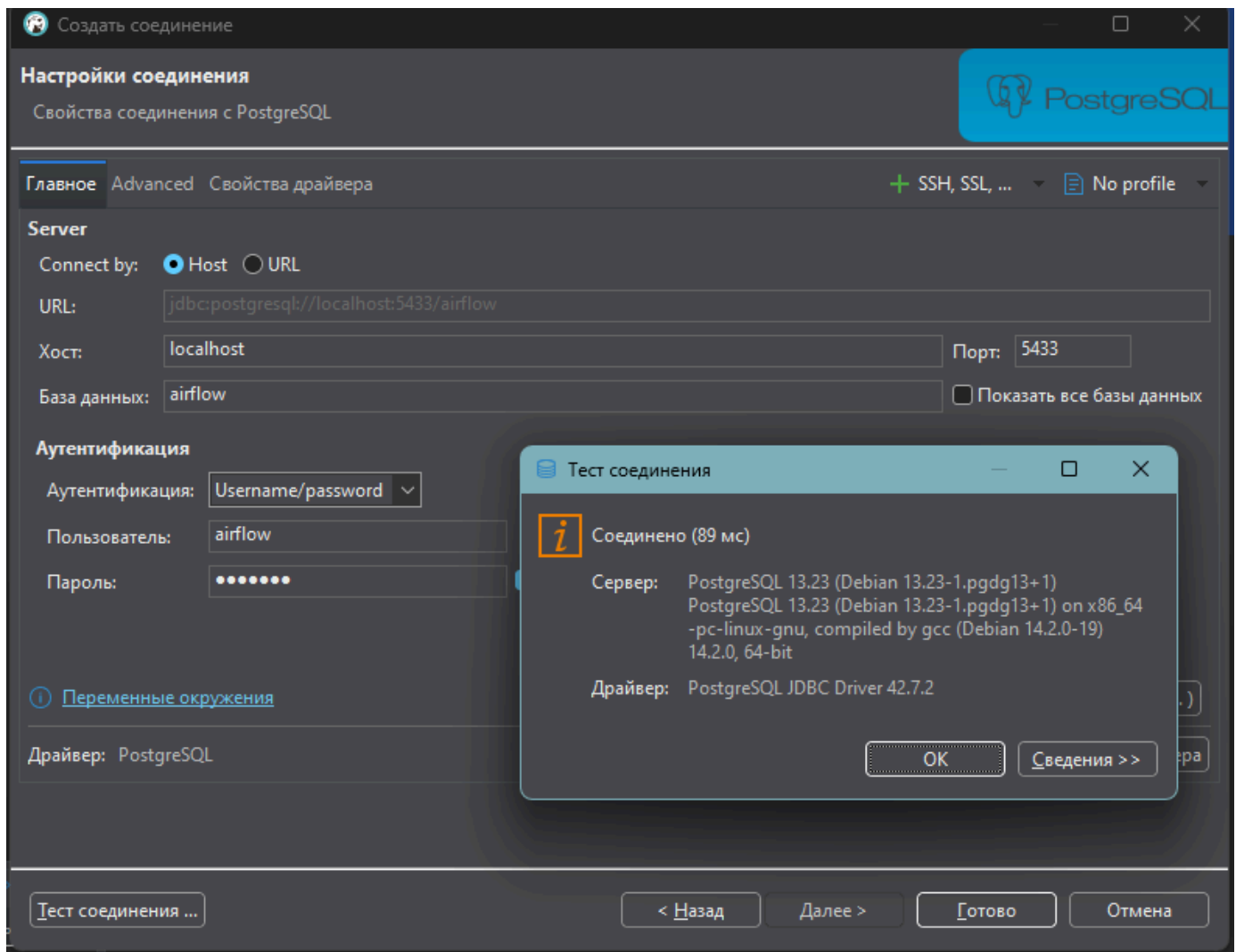


Рис. 2. Тест соединения с `airflow@localhost:5433` — Соединено, 89 мс.

Архитектура слоёв:

- LOGS — единственная таблица `logs.etl_log` с историей всех запусков и стадий;
- DS — детальный слой, шесть типизированных таблиц, в каждой первичный ключ согласно требованию;
- DM — заготовлен под витрины задач 1.2/1.3, в текущей задаче не используется.

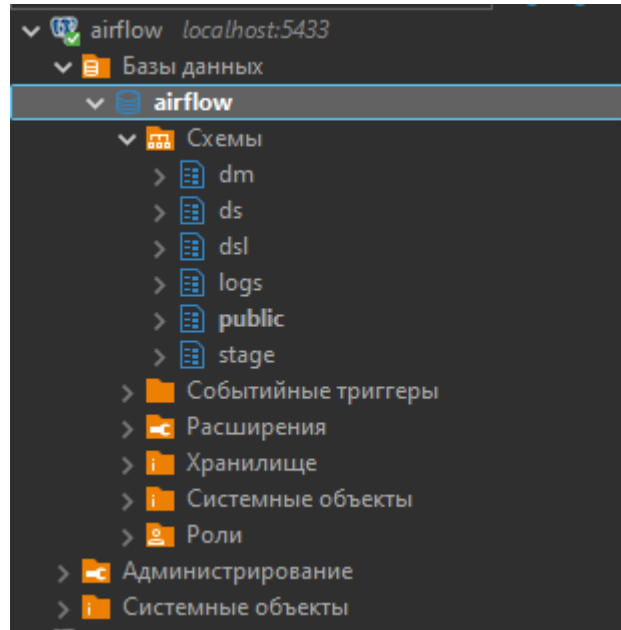


Рис. 3. Дерево схем БД airflow в DBeaver: dm, ds, logs.

3. Структура проекта

hw_coursework_task1/

├─ data/ шесть исходных CSV

| └─ ft_balance_f.csv

| └─ ft_posting_f.csv

| └─ md_account_d.csv

| └─ md_currency_d.csv

| └─ md_exchange_rate_d.csv

| └─ md_ledger_account_s.csv

├─ sql/

| └─ 01_ddl.sql схемы ds, logs (и dm для следующих задач)

| └─ 02_procedures.sql процедуры для задач 1.2 в 1.3

├─ python/

| └─ db.py параметры подключения

| └─ load_csv.py основной ETL — задача 1.1

├─ talend/

| └─ INSTRUCTION.md альтернативная сборка потока в Talend Open Studio

└─ out/ результаты экспорта (для задачи 1.4)

4. DDL: схемы и таблицы (sql/01_ddl.sql)

Скрипт идемпотентен — все CREATE сделаны через IF NOT EXISTS, поэтому DDL можно применять повторно. Каждый первичный ключ соответствует требованиям задания.

Таблица логов logs.etl_log спроектирована универсальной: в неё пишутся события всех ETL-стадий (load_csv_to_ds, seed_balance_2017_12_31 и процедуры расчётов витрин из последующих задач). Колонка extra хранит произвольную текстовую статистику — дату расчёта, длительность, имя CSV-файла или текст ошибки.

-- DDL: схемы DS, LOGS, DM и все таблицы курсовой работы.

```
CREATE SCHEMA IF NOT EXISTS ds;
```

```
CREATE SCHEMA IF NOT EXISTS logs;
```

```
CREATE SCHEMA IF NOT EXISTS dm;
```

-- Таблица логов ETL/процедур.

```
CREATE TABLE IF NOT EXISTS logs.etl_log (  
  log_id BIGSERIAL PRIMARY KEY,  
  process TEXT NOT NULL,  
  object_name TEXT,  
  event TEXT NOT NULL,  
  started_at TIMESTAMP,  
  finished_at TIMESTAMP,  
  rows_affected BIGINT,  
  extra TEXT,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

-- DS: детальный слой.

```
CREATE TABLE IF NOT EXISTS ds.ft_balance_f (  
  on_date DATE NOT NULL,  
  account_rk BIGINT NOT NULL,  
  currency_rk BIGINT,  
  balance_out NUMERIC,  
  CONSTRAINT pk_ft_balance_f PRIMARY KEY (on_date, account_rk)
```

);

CREATE TABLE IF NOT EXISTS ds.ft_posting_f (

oper_date DATE NOT NULL,

credit_account_rk BIGINT NOT NULL,

debet_account_rk BIGINT NOT NULL,

credit_amount NUMERIC,

debet_amount NUMERIC

);

CREATE TABLE IF NOT EXISTS ds.md_account_d (

data_actual_date DATE NOT NULL,

data_actual_end_date DATE NOT NULL,

account_rk BIGINT NOT NULL,

account_number VARCHAR(20) NOT NULL,

char_type VARCHAR(1) NOT NULL,

currency_rk BIGINT NOT NULL,

currency_code VARCHAR(3) NOT NULL,

CONSTRAINT pk_md_account_d PRIMARY KEY (data_actual_date, account_rk)

);

CREATE TABLE IF NOT EXISTS ds.md_currency_d (

currency_rk BIGINT NOT NULL,

data_actual_date DATE NOT NULL,

data_actual_end_date DATE,

currency_code VARCHAR(3),

code_iso_char VARCHAR(3),

CONSTRAINT pk_md_currency_d PRIMARY KEY (currency_rk, data_actual_date)

);

CREATE TABLE IF NOT EXISTS ds.md_exchange_rate_d (

data_actual_date DATE NOT NULL,

data_actual_end_date DATE,

```

currency_rk BIGINT NOT NULL,
reduced_cource NUMERIC,
code_iso_num VARCHAR(3),
CONSTRAINT pk_md_exchange_rate_d PRIMARY KEY (data_actual_date, currency_rk)
);

CREATE TABLE IF NOT EXISTS ds.md_ledger_account_s (
chapter CHAR(1),
chapter_name VARCHAR(16),
section_number INTEGER,
section_name VARCHAR(22),
subsection_name VARCHAR(21),
ledger1_account INTEGER,
ledger1_account_name VARCHAR(47),
ledger_account INTEGER NOT NULL,
ledger_account_name VARCHAR(153),
characteristic CHAR(1),
start_date DATE NOT NULL,
end_date DATE,
CONSTRAINT pk_md_ledger_account_s PRIMARY KEY (ledger_account, start_date)
);

```

5. ETL-процесс на Python (python/load_csv.py)

Логика загрузки:

- подключение по параметрам из db.py — host, port, db, user, password;
- запись в logs.etl_log события START всего процесса с описанием плана загрузки;
- пауза 5 секунд (требование задания);
- последовательно по каждому файлу: создание TEMP-таблицы с text-колонками по заголовку CSV, COPY данных из файла, INSERT в ds.* по правилу обновления для каждой таблицы, фиксация START/END в логах с длительностью и количеством строк;
- запись итогового события END с общей длительностью.

Обработка ошибок: при исключении на любом файле выполняется ROLLBACK транзакции файла, в logs.etl_log пишется событие ERROR с текстом исключения, и исключение пробрасывается вверх — процесс падает явно, без «тихих» потерь данных.

Обработка кодировки: файл сначала читается в бинарном режиме, затем декодируется как UTF-8; при ошибке декодирования (битый байт 0x98 в md_currency_d.csv) делается фоллбек на latin-1. После этого данные передаются в COPY уже как валидный UTF-8 поток.

Реализация Update strategy под каждую таблицу:

- ft_balance_f — INSERT ... ON CONFLICT (on_date, account_rk) DO UPDATE;
- ft_posting_f — TRUNCATE TABLE + INSERT (PK отсутствует, всегда полная загрузка);
- md_account_d — INSERT ... ON CONFLICT (data_actual_date, account_rk) DO UPDATE;
- md_currency_d — INSERT ... ON CONFLICT (currency_rk, data_actual_date) DO UPDATE;
- md_exchange_rate_d — INSERT ... ON CONFLICT (data_actual_date, currency_rk) DO UPDATE;
- md_ledger_account_s — INSERT ... ON CONFLICT (ledger_account, start_date) DO UPDATE.

В CSV-файлах справочников встречаются дублирующие записи по одной и той же паре PK. PostgreSQL не позволяет ON CONFLICT обновлять одну и ту же строку в рамках одного INSERT, поэтому выполняется предварительная дедупликация через DISTINCT ON по ключу — берётся первая встретившаяся запись.

Парсинг дат для каждого источника:

- ft_balance_f.csv — to_date(ON_DATE, 'DD.MM.YYYY');
- ft_posting_f.csv — to_date(OPER_DATE, 'DD-MM-YYYY');
- md_*.csv — поля DATA_ACTUAL_DATE/DATA_ACTUAL_END_DATE/START_DATE/END_DATE приходят в ISO-формате YYYY-MM-DD и приводятся прямым castом ::date.

5.1. Параметры подключения (python/db.py)

```
"""Параметры подключения к Postgres."""
```

```
import os
```

```
DB_CONFIG = {
```

```
"host": os.getenv("PGHOST", "localhost"),
```

```
"port": int(os.getenv("PGPORT", "5433")),
```

```
"dbname": os.getenv("PGDATABASE", "airflow"),
```

```
"user": os.getenv("PGUSER", "airflow"),
```

```
"password": os.getenv("PGPASSWORD", "airflow"),
```



```
}
```

5.2. Основной скрипт (python/load_csv.py)

"""Задача 1.1. Загрузка CSV в детальный слой DS с логированием в logs.etl_log.

Алгоритм:

1. Логируем событие START всего процесса.
2. Пауза 5 секунд (по требованию задания).
3. По каждому файлу:
 - START строки в логе,
 - COPY во временную stage-таблицу (TEMP),
 - INSERT ... ON CONFLICT DO UPDATE в ds.* (для ft_posting_f - TRUNCATE+INSERT),
 - END строки с количеством строк.
4. Логируем END всего процесса.

Запуск: python load_csv.py [path_to_data_dir]

"""

```
from __future__ import annotations
```

```
import os
```

```
import sys
```

```
import time
```

```
from datetime import datetime
```

```
import psycopg2
```

```
from db import DB_CONFIG
```

```
DATA_DIR = sys.argv[1] if len(sys.argv) > 1 else os.path.join(
```

```
os.path.dirname(__file__), "..", "data"
```

```
)
```

```
# (csv-имя, целевая таблица, формат даты для on_date/oper_date/etc, маппинг колонок)
```

```
# Все колонки CSV - в верхнем регистре, в Postgres - в нижнем.
```

```
# Стейдж заводим как TEMP с теми же типами text, чтобы не зависеть от формата дат.
```

```
LOAD_PLAN = [
```

```
# (csv, target, date_cols)
```

```
(
    "ft_balance_f.csv", "ds.ft_balance_f", {"on_date": "DD.MM.YYYY"}),
    ("ft_posting_f.csv", "ds.ft_posting_f", {"oper_date": "DD-MM-YYYY"}),
    ("md_account_d.csv", "ds.md_account_d", {"data_actual_date": None,
    "data_actual_end_date": None}),
    ("md_currency_d.csv", "ds.md_currency_d", {"data_actual_date": None,
    "data_actual_end_date": None}),
    ("md_exchange_rate_d.csv", "ds.md_exchange_rate_d", {"data_actual_date": None,
    "data_actual_end_date": None}),
    ("md_ledger_account_s.csv", "ds.md_ledger_account_s", {"start_date": None,
    "end_date": None}),
]
```

Каждой целевой таблице - SQL для записи или замены из stage.

```
UPSERT_SQL = {
    "ds.ft_balance_f": """
INSERT INTO ds.ft_balance_f (on_date, account_rk, currency_rk, balance_out)
SELECT DISTINCT ON (to_date("ON_DATE",'DD.MM.YYYY'), "ACCOUNT_RK"::bigint)
to_date("ON_DATE",'DD.MM.YYYY'),
"ACCOUNT_RK"::bigint,
NULLIF("CURRENCY_RK",)::bigint,
NULLIF("BALANCE_OUT",)::numeric
FROM stg
WHERE "ON_DATE" IS NOT NULL AND "ACCOUNT_RK" IS NOT NULL
ON CONFLICT (on_date, account_rk) DO UPDATE
SET currency_rk = EXCLUDED.currency_rk,
balance_out = EXCLUDED.balance_out;
""",
    # ft_posting_f - без ПК, всегда полная загрузка.
    "ds.ft_posting_f": """
TRUNCATE TABLE ds.ft_posting_f;
```

```

INSERT INTO ds.ft_posting_f
(oper_date, credit_account_rk, debet_account_rk, credit_amount, debet_amount)
SELECT to_date("OPER_DATE",'DD-MM-YYYY'),
"CREDIT_ACCOUNT_RK"::bigint,
"DEBET_ACCOUNT_RK"::bigint,
NULLIF("CREDIT_AMOUNT",)::numeric,
NULLIF("DEBET_AMOUNT",)::numeric
FROM stg
WHERE "OPER_DATE" IS NOT NULL;
""",
"ds.md_account_d": """"
INSERT INTO ds.md_account_d (data_actual_date, data_actual_end_date,
account_rk, account_number, char_type,
currency_rk, currency_code)
SELECT DISTINCT ON ("DATA_ACTUAL_DATE","ACCOUNT_RK")
"DATA_ACTUAL_DATE"::date,
"DATA_ACTUAL_END_DATE"::date,
"ACCOUNT_RK"::bigint,
"ACCOUNT_NUMBER",
"CHAR_TYPE",
"CURRENCY_RK"::bigint,
"CURRENCY_CODE"
FROM stg
ON CONFLICT (data_actual_date, account_rk) DO UPDATE
SET data_actual_end_date = EXCLUDED.data_actual_end_date,
account_number = EXCLUDED.account_number,
char_type = EXCLUDED.char_type,
currency_rk = EXCLUDED.currency_rk,
currency_code = EXCLUDED.currency_code;

```

```

""",

"ds.md_currency_d": ""

INSERT INTO ds.md_currency_d (currency_rk, data_actual_date,
data_actual_end_date, currency_code, code_iso_char)
SELECT DISTINCT ON ("CURRENCY_RK","DATA_ACTUAL_DATE")
"CURRENCY_RK"::bigint,
"DATA_ACTUAL_DATE"::date,
NULLIF("DATA_ACTUAL_END_DATE",)::date,
"CURRENCY_CODE",
"CODE_ISO_CHAR"
FROM stg
ON CONFLICT (currency_rk, data_actual_date) DO UPDATE
SET data_actual_end_date = EXCLUDED.data_actual_end_date,
currency_code = EXCLUDED.currency_code,
code_iso_char = EXCLUDED.code_iso_char;

""",

"ds.md_exchange_rate_d": ""

INSERT INTO ds.md_exchange_rate_d (data_actual_date, data_actual_end_date,
currency_rk, reduced_cource, code_iso_num)
SELECT DISTINCT ON ("DATA_ACTUAL_DATE", "CURRENCY_RK")
"DATA_ACTUAL_DATE"::date,
NULLIF("DATA_ACTUAL_END_DATE",)::date,
"CURRENCY_RK"::bigint,
NULLIF("REDUCED_COURSE",)::numeric,
"CODE_ISO_NUM"
FROM stg
ON CONFLICT (data_actual_date, currency_rk) DO UPDATE
SET data_actual_end_date = EXCLUDED.data_actual_end_date,
reduced_cource = EXCLUDED.reduced_cource,

```

```

code_iso_num = EXCLUDED.code_iso_num;

""",

"ds.md_ledger_account_s": ""

INSERT INTO ds.md_ledger_account_s (chapter, chapter_name, section_number,
section_name, subsection_name, ledger1_account, ledger1_account_name,
ledger_account, ledger_account_name, characteristic, start_date, end_date)
SELECT DISTINCT ON ("LEDGER_ACCOUNT","START_DATE")
"CHAPTER",
"CHAPTER_NAME",
NULLIF("SECTION_NUMBER","")::int,
"SECTION_NAME",
"SUBSECTION_NAME",
NULLIF("LEDGER1_ACCOUNT","")::int,
"LEDGER1_ACCOUNT_NAME",
"LEDGER_ACCOUNT"::int,
"LEDGER_ACCOUNT_NAME",
"CHARACTERISTIC",
"START_DATE"::date,
NULLIF("END_DATE","")::date
FROM stg

ON CONFLICT (ledger_account, start_date) DO UPDATE

SET chapter = EXCLUDED.chapter,
chapter_name = EXCLUDED.chapter_name,
section_number = EXCLUDED.section_number,
section_name = EXCLUDED.section_name,
subsection_name = EXCLUDED.subsection_name,
ledger1_account = EXCLUDED.ledger1_account,
ledger1_account_name = EXCLUDED.ledger1_account_name,
ledger_account_name = EXCLUDED.ledger_account_name,

```

```

characteristic = EXCLUDED.characteristic,

end_date = EXCLUDED.end_date;

""",

}

def get_csv_header_columns(path: str) в list[str]:

with open(path, "rb") as f:

line = f.readline()

try:

header = line.decode("utf-8").rstrip("\r\n")

except UnicodeDecodeError:

header = line.decode("latin-1").rstrip("\r\n")

return header.split(";")

def log_event(cur, process: str, object_name: str, event: str,

started_at: datetime | None = None,

finished_at: datetime | None = None,

rows: int | None = None, extra: str | None = None) в int:

cur.execute(

"""

INSERT INTO logs.etl_log(process, object_name, event,

started_at, finished_at, rows_affected, extra)

VALUES (%s, %s, %s, %s, %s, %s, %s)

RETURNING log_id

""",

(process, object_name, event, started_at, finished_at, rows, extra),

)

return cur.fetchone()[0]

def read_csv_text(path: str) в "io.StringIO":

"""Читает CSV в StringIO (UTF-8). Если файл невалидный UTF-8 -

декодируем как latin-1 (устойчивость к одиночным «битым» байтам).
```

```
"""
```

```
import io
```

```
with open(path, "rb") as fh:
```

```
raw = fh.read()
```

```
try:
```

```
text = raw.decode("utf-8")
```

```
except UnicodeDecodeError:
```

```
text = raw.decode("latin-1")
```

```
return io.StringIO(text)
```

```
def load_one(conn, csv_name: str, target: str) в int:
```

```
"""Грузит один CSV в target. Возвращает количество строк."""
```

```
csv_path = os.path.join(DATA_DIR, csv_name)
```

```
columns = get_csv_header_columns(csv_path)
```

```
with conn.cursor() as cur:
```

```
# 1. Стейдж - временная таблица с text-колонками по заголовку.
```

```
cur.execute("DROP TABLE IF EXISTS stg;")
```

```
cols_def = ", ".join(f"{c}" text' for c in columns)
```

```
cur.execute(f"CREATE TEMP TABLE stg ({cols_def}) ON COMMIT DROP;")
```

```
# 2. COPY: читаем в память с авто-выбором кодировки.
```

```
buf = read_csv_text(csv_path)
```

```
cur.copy_expert(
```

```
sql=("COPY stg FROM STDIN WITH "
```

```
"(FORMAT CSV, HEADER true, DELIMITER ';', NULL ")"),
```

```
file=buf,
```

```
)
```

```
# 3. UPSERT/REPLACE в ds.*.
```

```
cur.execute(UPSERT_SQL[target])
```

```
cur.execute(f"SELECT count(*) FROM {target};")
```

```
total_rows = cur.fetchone()[0]
```

```

return total_rows

def main() в None:

print(f"DATA_DIR={DATA_DIR}")

started = datetime.now()

with psycopg2.connect(**DB_CONFIG) as conn:

with conn.cursor() as cur:

log_event(cur, "load_csv_to_ds", "ALL",

"START", started_at=started,

extra=f"plan={[t for _,t,_ in LOAD_PLAN]}")

conn.commit()

print("[INFO] Логирована START всего процесса. Пауза 5 секунд...")

time.sleep(5)

for csv_name, target, _ in LOAD_PLAN:

t0 = datetime.now()

print(f"[LOAD] {csv_name:32s} в {target}")

with conn.cursor() as cur:

log_event(cur, "load_csv_to_ds", target, "START",

started_at=t0, extra=csv_name)

conn.commit()

try:

rows = load_one(conn, csv_name, target)

conn.commit()

except Exception as exc:

conn.rollback()

with conn.cursor() as cur:

log_event(cur, "load_csv_to_ds", target, "ERROR",

started_at=t0, finished_at=datetime.now(),

extra=str(exc)[:500])

conn.commit()

```



```

raise

t1 = datetime.now()

with conn.cursor() as cur:

log_event(cur, "load_csv_to_ds", target, "END",
started_at=t0, finished_at=t1,

rows=rows,

extra=f'duration_sec={{(t1-t0).total_seconds():.3f}}')

conn.commit()

print(f'[OK] {target} rows={rows} {(t1-t0).total_seconds():.2f}s')

finished = datetime.now()

with conn.cursor() as cur:

log_event(cur, "load_csv_to_ds", "ALL", "END",

started_at=started, finished_at=finished,

extra=f'duration_sec={{(finished-started).total_seconds():.2f}}')

conn.commit()

print(f'[DONE] Загрузка завершена за {(finished-started).total_seconds():.2f} сек.")

if __name__ == "__main__":

main()

```

6. Запуск и результаты

Перед запуском очищаем все шесть таблиц DS, чтобы убедиться, что данные приехали именно из ETL:

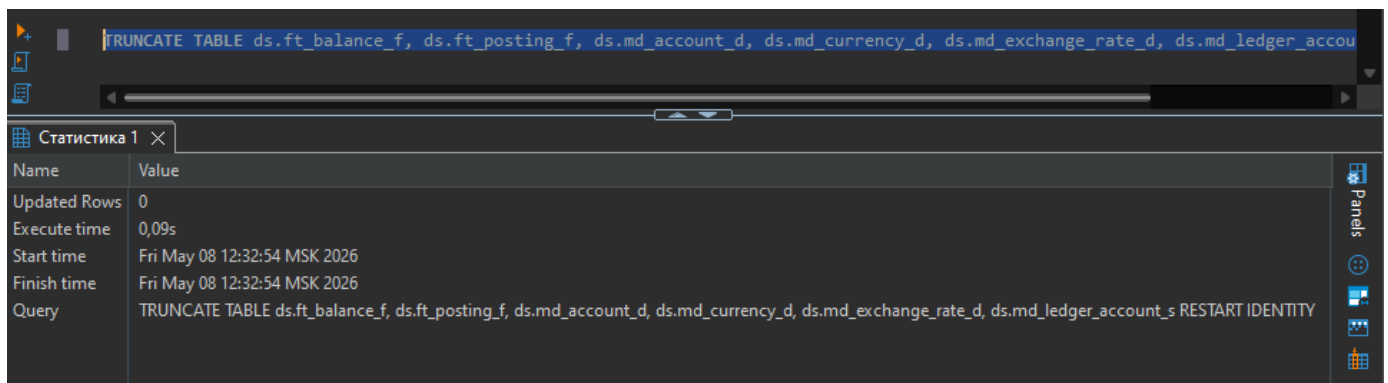


Рис. 4. `TRUNCATE TABLE` для всех шести таблиц `ds.*` (DBeaver; 0.09 с).

Запуск загрузчика:

```
cd python && python load_csv.py
```

Реальный вывод (фрагмент консоли):

```
C:\Users\maks\\Desktop\neoflex\hw_coursework_task1\python>python load_csv.py
DATA_DIR=C:\Users\maks\\Desktop\neoflex\hw_coursework_task1\python\..\data
[INFO] Логирована START всего процесса. Пауза 5 секунд...
[LOAD] ft_balance_f.csv -> ds.ft_balance_f
[OK] ds.ft_balance_f rows=114 0.01s
[LOAD] ft_posting_f.csv -> ds.ft_posting_f
[OK] ds.ft_posting_f rows=33892 0.12s
[LOAD] md_account_d.csv -> ds.md_account_d
[OK] ds.md_account_d rows=112 0.01s
[LOAD] md_currency_d.csv -> ds.md_currency_d
[OK] ds.md_currency_d rows=50 0.01s
[LOAD] md_exchange_rate_d.csv -> ds.md_exchange_rate_d
[OK] ds.md_exchange_rate_d rows=460 0.02s
[LOAD] md_ledger_account_s.csv -> ds.md_ledger_account_s
[OK] ds.md_ledger_account_s rows=18 0.01s
[DONE] Загрузка завершена за 5.22 сек.

C:\Users\maks\\Desktop\neoflex\hw_coursework_task1\python>
```

Рис. 5. Запуск `python load_csv.py` — все 6 файлов загружены, общее время 5.22 с (включая обязательную паузу 5 с).

Контрольный подсчёт строк в `ds.ft_balance_f` после загрузки:

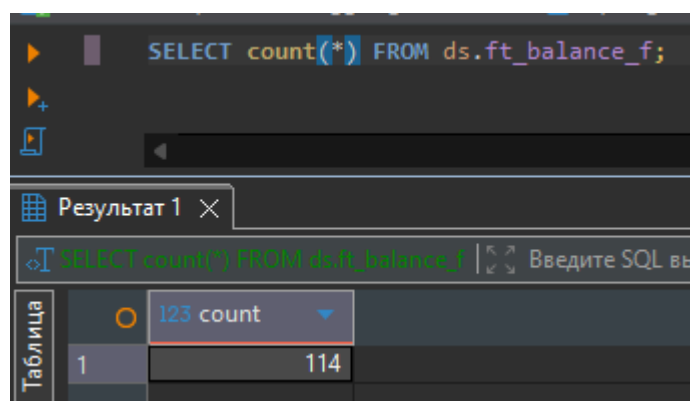


Рис. 6. `SELECT count(*) FROM ds.ft_balance_f = 114` — совпадает с числом строк в CSV.

Сводка по всем шести таблицам после загрузки:

таблица | count

-----+-----

ds.ft_balance_f | 114

ds.ft_posting_f | 33892

ds.md_account_d | 112

ds.md_currency_d | 50

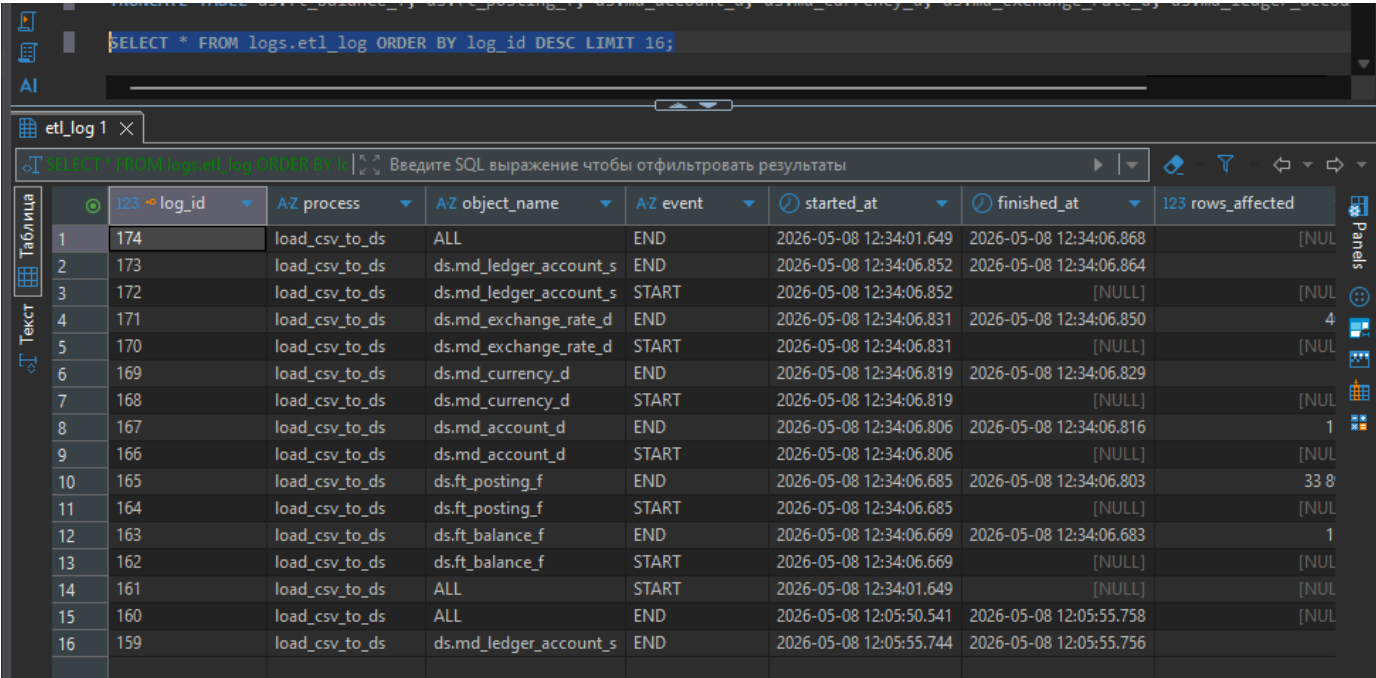
ds.md_exchange_rate_d | 460

ds.md_ledger_account_s | 18

7. Логирование

Все стадии прокидываются в logs.etl_log с фиксированным набором полей: process, object_name, event (START / END / ERROR), started_at, finished_at, rows_affected, extra. Это даёт полную трассировку запуска: сколько шёл процесс целиком, сколько шёл каждый файл, сколько строк попало в каждую целевую таблицу, и в случае ошибки — текст исключения. Между событием START всего процесса и его END видно реальный отрыв примерно в 5 секунд из-за обязательного таймера.

Реальная выборка из таблицы logs.etl_log после успешной загрузки (последние 16 событий):



The screenshot shows a database query interface. At the top, a SQL query is entered: `SELECT * FROM logs.etl_log ORDER BY log_id DESC LIMIT 16;`. Below the query, a table view displays the results. The table has columns: log_id, process, object_name, event, started_at, finished_at, and rows_affected. The results show 16 rows of log entries, ordered by log_id in descending order. The events are a mix of START and END for various processes and objects. The rows_affected column shows the number of rows affected by each event.

log_id	process	object_name	event	started_at	finished_at	rows_affected
174	load_csv_to_ds	ALL	END	2026-05-08 12:34:01.649	2026-05-08 12:34:06.868	[NUL]
173	load_csv_to_ds	ds.md_ledger_account_s	END	2026-05-08 12:34:06.852	2026-05-08 12:34:06.864	
172	load_csv_to_ds	ds.md_ledger_account_s	START	2026-05-08 12:34:06.852	[NULL]	[NUL]
171	load_csv_to_ds	ds.md_exchange_rate_d	END	2026-05-08 12:34:06.831	2026-05-08 12:34:06.850	4
170	load_csv_to_ds	ds.md_exchange_rate_d	START	2026-05-08 12:34:06.831	[NULL]	[NUL]
169	load_csv_to_ds	ds.md_currency_d	END	2026-05-08 12:34:06.819	2026-05-08 12:34:06.829	
168	load_csv_to_ds	ds.md_currency_d	START	2026-05-08 12:34:06.819	[NULL]	[NUL]
167	load_csv_to_ds	ds.md_account_d	END	2026-05-08 12:34:06.806	2026-05-08 12:34:06.816	1
166	load_csv_to_ds	ds.md_account_d	START	2026-05-08 12:34:06.806	[NULL]	[NUL]
165	load_csv_to_ds	ds.ft_posting_f	END	2026-05-08 12:34:06.685	2026-05-08 12:34:06.803	33 8
164	load_csv_to_ds	ds.ft_posting_f	START	2026-05-08 12:34:06.685	[NULL]	[NUL]
163	load_csv_to_ds	ds.ft_balance_f	END	2026-05-08 12:34:06.669	2026-05-08 12:34:06.683	1
162	load_csv_to_ds	ds.ft_balance_f	START	2026-05-08 12:34:06.669	[NULL]	[NUL]
161	load_csv_to_ds	ALL	START	2026-05-08 12:34:01.649	[NULL]	[NUL]
160	load_csv_to_ds	ALL	END	2026-05-08 12:05:50.541	2026-05-08 12:05:55.758	[NUL]
159	load_csv_to_ds	ds.md_ledger_account_s	END	2026-05-08 12:05:55.744	2026-05-08 12:05:55.756	

Рис. 7. logs.etl_log — START/END для ALL и для каждой из шести таблиц, с rows_affected.

Видно, что записи идут парами START/END. У строки END для ds.ft_posting_f rows_affected = 33 892, у ds.md_exchange_rate_d = 460 и т.д. Между log_id START всего процесса и log_id START первого файла проходят те самые 5 секунд таймера.

8. Демонстрация режима «Запись или замена»

Задание требует показать, что повторный запуск ETL обновляет уже существующие записи по их РК. Сценарий выполнен на счёте account_rk=13560.

Шаг 1. Состояние строки после первой загрузки — balance_out = 1 055 629,43:

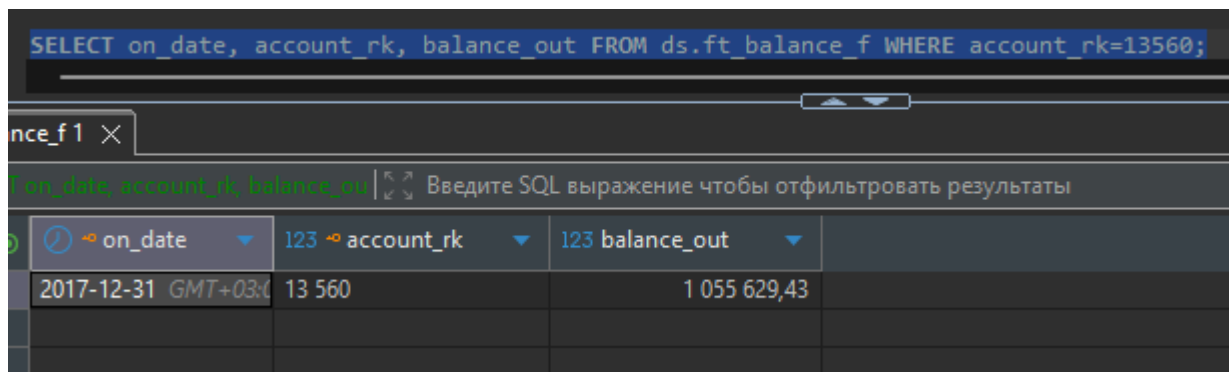


Рис. 8. Исходное значение `balance_out` для `account_rk=13560`.

Шаг 2. В файле `data/ft_balance_f.csv` (Excel/Notepad) меняем значение `balance_out` у нужной строки:

	A	B	C	D
1	ON_DATE	ACCOUNT_RK	CURRENCY_RK	BALANCE_OUT
2	31.12.2017	36237725	35	38318.13
3	31.12.2017	24656	35	80533.62
4	31.12.2017	18849846	34	63891.96
5	31.12.2017	1972647	34	5087732.1
6	31.12.2017	34157174	34	7097806.9
7	31.12.2017	48829156	34	87620.47
8	31.12.2017	13905	34	129554
9	31.12.2017	17244	34	2025852.49
10	31.12.2017	34156787	31	32640.33
11	31.12.2017	17132	34	20011371.58
12	31.12.2017	13560	34	1055629.43
13	31.12.2017	17439	34	130813.42
14	31.12.2017	17434	34	5008133.82
15	31.12.2017	13630	34	200079650.12
16	31.12.2017	277937699	34	35341.17
17	31.12.2017	13906	30	194729.89
18	31.12.2017	14136	31	44626.61
19	31.12.2017	13904	34	5541.83
20	31.12.2017	34157147	35	13562.04
21	31.12.2017	14138	44	20003622.76
22	31.12.2017	34156756	34	99438.49
23	31.12.2017	81940552	37	3462.19

Рис. 9. Правка строки 12 файла `ft_balance_f.csv` — `account_rk=13560`.

Шаг 3. Повторный запуск `python load_csv.py` (без чистки таблиц) — все 114 строк `ft_balance_f` проходят через UPSERT:

```

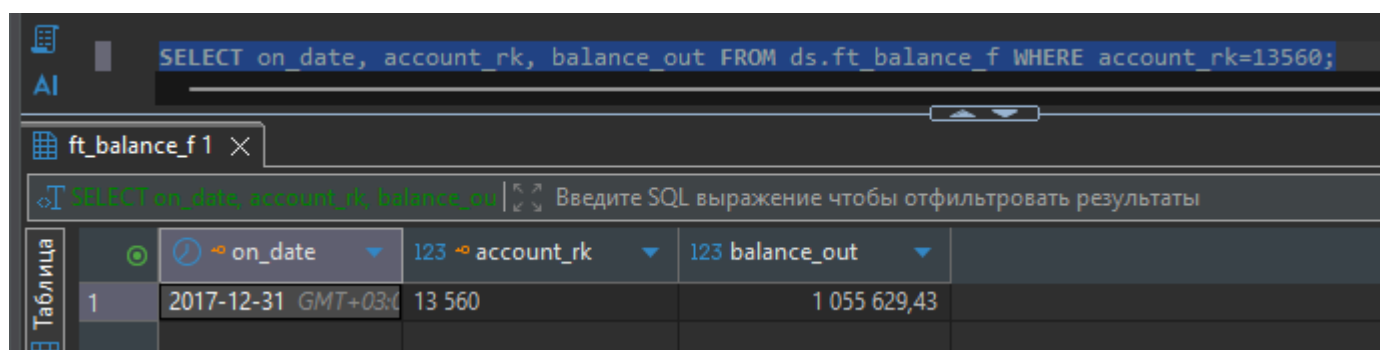
C:\Users\maksi\Desktop\neoflex\hw_coursework_task1\python>python load_csv.py
DATA_DIR=C:\Users\maksi\Desktop\neoflex\hw_coursework_task1\python\..\data
[INFO] Логирована START всего процесса. Пауза 5 секунд...
[LOAD] ft_balance_f.csv -> ds.ft_balance_f
[OK] ds.ft_balance_f rows=114 0.01s
[LOAD] ft_posting_f.csv -> ds.ft_posting_f
[OK] ds.ft_posting_f rows=33892 0.12s
[LOAD] md_account_d.csv -> ds.md_account_d
[OK] ds.md_account_d rows=112 0.01s
[LOAD] md_currency_d.csv -> ds.md_currency_d
[OK] ds.md_currency_d rows=50 0.01s
[LOAD] md_exchange_rate_d.csv -> ds.md_exchange_rate_d
[OK] ds.md_exchange_rate_d rows=460 0.02s
[LOAD] md_ledger_account_s.csv -> ds.md_ledger_account_s
[OK] ds.md_ledger_account_s rows=18 0.01s
[DONE] Загрузка завершена за 5.22 сек.

C:\Users\maksi\Desktop\neoflex\hw_coursework_task1\python>

```

Рис. 10. Повторный запуск ETL — те же rows=114, время загрузки те же ~5.2 с с учётом таймера.

Шаг 4. Состояние строки после повторного запуска — значение обновилось:



on_date	account_rk	balance_out
2017-12-31 GMT+03:00	13 560	1 055 629,43

Рис. 11. После повторной загрузки balance_out по тому же ключу обновился без дублей.

Сумма по тому же ключу обновилась без дублирования строки. Это и есть запрошенный режим «Запись или замена», обеспеченный выражением INSERT ... ON CONFLICT (on_date, account_rk) DO UPDATE.

9. Альтернативная реализация в Talend Open Studio

В архиве курса предоставлен Talend Open Studio for DI 7.1.1 + JDK 8. Среда распакована в каталог talend_install/extracted, JDK 8 уже установлен на машине и используется. Запускается исполняемым TOS_DI-win-x86_64.exe. Подробная инструкция по сборке job-а лежит в talend/INSTRUCTION.md.

9.1. Подключение к Postgres из Talend

Создаём метаданные DB Connection (Repository → Metadata → Db Connections). Изначально по ошибке в поле «Сервер» был указан airflow вместо localhost — Talend выдал «Ошибка соединения»:

Соединение с БД

Соединение новой БД с репозиторием - Шаг 2/2

⚠ Ошибка соединения. Вы должны изменить настройки базы данных.

Тип БД PostgreSQL

Версия БД	v9 and later
Строка соединения	jdbc:postgresql://airflow:5433/airflow
Логин	airflow
Пароль	••••••••
Сервер	airflow
Порт	5433
DataBase	airflow
Схема	ds

Test connection v

Экспортировать как контекст Восстановить контекст

[How to install a driver](#)

< Back Next > Finish Cancel

Рис. 12. Ошибка соединения: в поле Сервер ошибочно указано airflow.

Кроме того, в стандартной поставке TOS DI 7.1.1 отсутствует JDBC-драйвер postgresql-42.2.5.jar, из-за чего тест соединения падает с `MissingDriverException`:

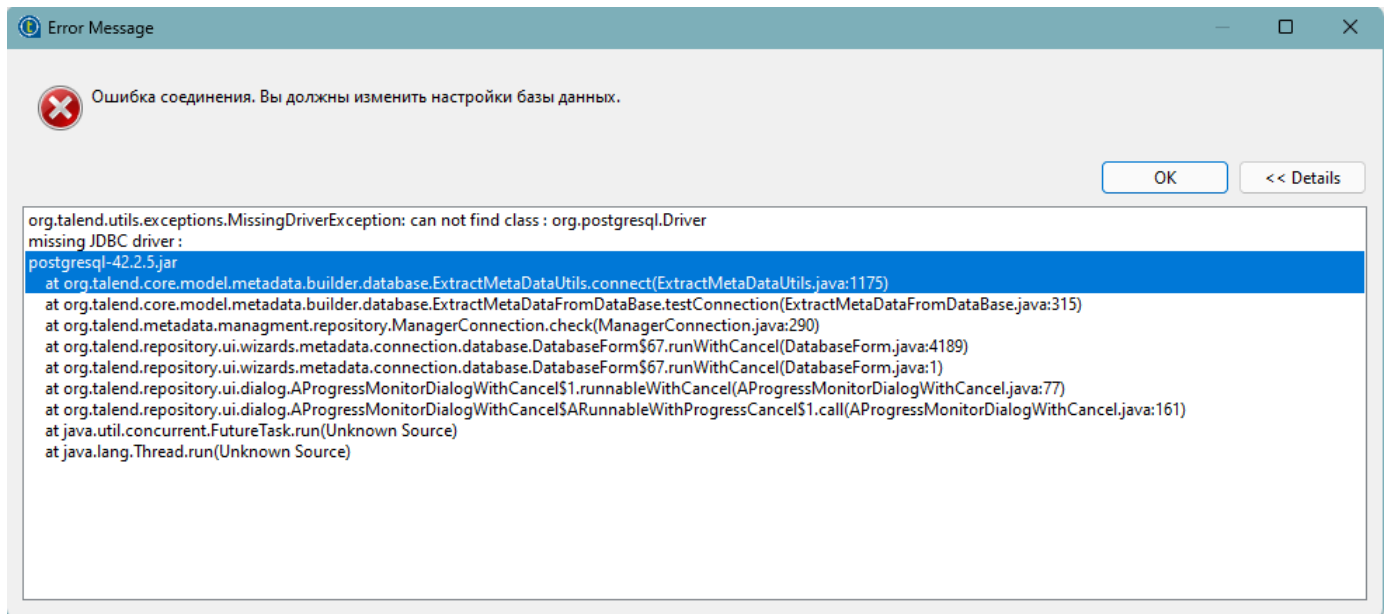


Рис. 13. *MissingDriverException: org.postgresql.Driver* — драйвер нужно добавить вручную.

Скачали `postgresql-42.2.5.jar` и положили в `configuration/.m2/repository/org/postgresql/postgresql/42.2.5`. После исправления хоста на `localhost` и подкладки `jar` тест соединения проходит:

Соединение с БД

Соединение новой БД с репозиторием - Шаг 2/2

Вы должны нажать кнопку "Проверить", чтобы проверить настройки базы данных

Тип БД PostgreSQL

Версия БД v9 and later

Строка соединения jdbc:postgresql://localhost:5433/airflow

Логин airflow

Пароль

Сервер localhost

Порт 5433

DataBase airflow

Схема ds

Test connection v

Экспортировать как контекст Восстановить контекст

[How to install a driver](#)

< Back Next > Finish Cancel

Рис. 14. Корректные параметры соединения pg_dwh: localhost:5433, БД airflow, схема ds.

9.2. Импорт схем таблиц

В Talend через Retrieve schema подтягиваем структуру таблицы ds.ft_balance_f (для демонстрации Talend-ветки делаем загрузку только этого файла, остальные пять делает Python):

New Schema in connection "pg_dwh"

Add a Schema on repository



Select Schema to create

Name Filter:

Имя	Тип	Номер столбца	Creation status
▼ airflow	CATALOG		
▼ ds	SCHEMA		
☒ ft_balance_f	TABLE	4	Успешно
☐ ft_posting_f	TABLE		
☐ md_account_d	TABLE		
☐ md_currency_d	TABLE		
☐ md_exchange_rate_d	TABLE		
☐ md_ledger_account_s	TABLE		

Выбрать все Select None Проверить соединение

< Back

Next >

Finish

Cancel

Рис. 15. Retrieve schema — выбираем *ds.ft_balance_f*.

New Schema in connection "pg_dwh"
Add a Schema on repository

Схема
ft_balance_f

Имя: ft_balance_f
Комментарий:
Тип: TABLE

Основанный на таблице: ft_balance_f  Восстановить схему Предположить схему

Use the "Retrieve Schema" button to replace the current Schema by the table based Schema

Схема

Колонка	Db Column	Кл...	Тип БД	Тип	☒	N..	Date Pat...	Дли...	Пре...	D...	Ко...
 on_date	on_date	☒	DATE	Date	☐		"dd-M...	13	0		
 account_rk	account_rk	☒	INT8	long	☐			19	0		
currency_rk	currency_rk	☐	INT8	Long	☒			19	0		
balance_out	balance_out	☐	NUME...	Big...	☒			131...	0		

Добавить схему
Remove Schema

< Back Next > Finish Cancel

Рис. 16. Импортированная схема *ft_balance_f* с PK на *on_date+account_rk* и типами *Date/Long/Long/BigDecimal*.

9.3. Метаданные File delimited

Создаём File delimited для *ft_balance_f.csv*. На шаге 3 важно правильно задать настройки разбора. Сначала Header=1 без галочки «установить заголовок» — заголовок остаётся в превью отдельной строкой:

File - Step 3 of 4

Add a Metadata File on repository
Define the setting of the parse job



Настройки файла

Кодировка: UTF-8

Разделитель полей: Semicolon Corresponding Character: ";"

Разделитель строк: Standard EOL Corresponding Character: "\\n"

Escape Char Settings

☐ CSV ☒ Разделенный

Escape Char: Пустой

Text Enclosure: Пустой

☐ Split row before field

Rows To Skip

Определите следующие параметры, если какие-либо строки должны быть п

Header ☒ 1

Footer ☐

☐ Skip empty row

Ограничение строк

Если количество строк должно быть ограничено, определите это количеств

Ограничение ☐

Предпросмотр

Выход

☐ Установить строку заголовка в качестве имен столбцов

Refresh Preview

Колонка 0	Колонка 1	Колонка 2	Колонка 3
ON_DATE	ACCOUNT_RK	CURRENCY_RK	BALANCE_OUT
31.12.2017	36237725	35	38318.13
31.12.2017	24656	35	80533.62
31.12.2017	18849846	34	63891.96
31.12.2017	1972647	34	5087732.1
31.12.2017	34157174	34	7097806.9

Экспортировать как контекст

Восстановить контекст

< Back

Next >

Finish

Cancel

Рис. 17. Step 3 — UTF-8, разделитель Semicolon, Header=1.

Если включить галочку «Установить строку заголовка в качестве имён столбцов», Talend ставит Header=2 и пытается использовать данные как заголовок — этого не нужно:

File - Step 3 of 4
Add a Metadata File on repository
Define the setting of the parse job

Настройки файла

Кодировка: UTF-8

Разделитель полей: Semicolon Corresponding Character: ";"

Разделитель строк: Standard EOL Corresponding Character: "\n"

Escape Char Settings

☐ CSV ☒ Разделенный

Escape Char: Пустой

Text Enclosure: Пустой

☐ Split row before field

Rows To Skip

Определите следующие параметры, если какие-либо строки должны быть проигнорированы

Header ☒ 2

Footer ☐

☐ Skip empty row

Ограничение строк

Если количество строк должно быть ограничено, определите это количество.

Ограничение ☐

Предпросмотр Выход

☒ Установить строку заголовка в качестве имен столбцов Refresh Preview

31.12.2017	36237725	35	38318.13
31.12.2017	24656	35	80533.62
31.12.2017	18849846	34	63891.96
31.12.2017	1972647	34	5087732.1
31.12.2017	34157174	34	7097806.9
31.12.2017	48829156	34	87620.47
31.12.2017	13905	34	129554
31.12.2017	17244	34	2025852.49

Экспортировать как контекст Восстановить контекст

< Back Next > Finish Cancel

Рис. 18. Ошибочный режим Header=2 — первая строка данных трактуется как заголовок.

При неверной настройке Talend на шаге 4 формирует «битые» имена колонок _1_12_2017 / _6237725 / _5 / _8318_13 и угадывает типы Integer/Float — неприемлемо:

File - Step 4 of 4

Add a Schema on repository
Define the Schema

Имя

metadata

Комментарий

Схема

Click to update schema preview

Guess

Описание схемы

Колонка	Кл...	Тип	☒ N..	Date Pattern (Ctrl+Space a...	Длина	Precision	Default	Комментарий
_1_12_2017	☐	String	☒		10	0		
_6237725	☐	Integer	☒		9	0		
_5	☐	Integer	☒		2	0		
_8318_13	☐	Float	☒		12	7		

< Back

Next >

Finish

Cancel

Рис. 19. Step 4 с битыми именами столбцов — следствие неверного Header.

Возвращаемся, ставим Header=1, отключаем галочку, на шаге 4 жмём Guess. Имена корректные, но Talend выводит типы Integer/Float — для устойчивого парсинга через tMap переключаем все колонки в String:

File - Step 4 of 4
 Add a Schema on repository
 Define the Schema

Имя:

Комментарий:

Схема

Click to update schema preview

Описание схемы

Колонка	Кл...	Тип	☒ N..	Date Pattern (Ctrl+Space a...	Длина	Precision	Default	Комментарий
ON_DATE	☐	String	☒		10	0		
ACCOUNT_RK	☐	Integer	☒		9	0		
CURRENCY_RK	☐	Integer	☒		2	0		
BALANCE_OUT	☐	Float	☒		12	7		

Рис. 20. Step 4 — корректные имена *ON_DATE/ACCOUNT_RK/CURRENCY_RK/BALANCE_OUT*, типы по умолчанию.

File - Step 4 of 4

Add a Schema on repository

Define the Schema

Имя

metadata

Комментарий

Схема

Click to update schema preview

Guess

Описание схемы

Колонка	Кл...	Тип	☒	N..	Date Pattern (Ctrl+Space a...	Длина	Precision	Default	Комментарий
ON_DATE	☐	String	☒			10	0		
ACCOUNT_RK	☐	String	☒				0		
CURRENCY_RK	☐	String	☒				0		
BALANCE_OUT	☐	String	☒				7		

< Back

Next >

Finish

Cancel

Рис. 21. Все четыре колонки переведены в String — типизация выполняется в tMap.

File - Step 4 of 4

Add a Schema on repository
Define the Schema

Имя

Комментарий

Схема

Click to update schema preview

Описание схемы

Колонка	Кл...	Тип	☒ N..	Date Pattern (Ctrl+Space a...	Длина	Precision	Default	Комментарий
ON_DATE	☐	String	☒		10	0		
ACCOUNT_RK	☐	String	☒			0		
CURRENCY_RK	☐	String	☒			0		
BALANCE_OUT	☐	String	☒			7		

Рис. 22. Имя метаданных задано как `csv_ft_balance_f`.

9.4. Сборка `job_load_ft_balance_f`

Job состоит из трёх компонентов: `tFileInputDelimited (csv_ft_balance_f) → tMap_1 → tDBOutput («ft_balance_f», PostgreSQL)`:

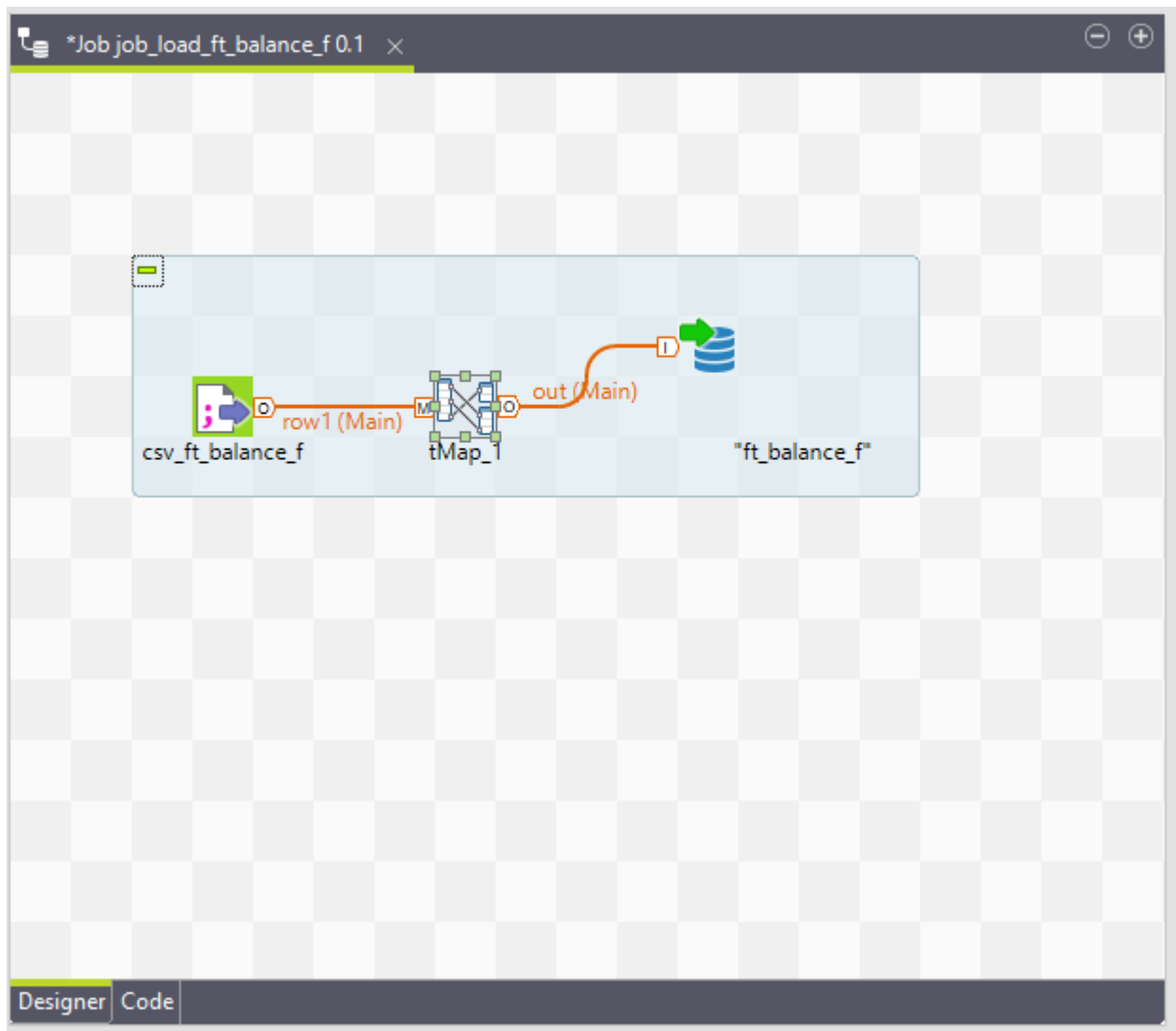


Рис. 23. Канва `job_load_ft_balance_f` в Talend Open Studio.

В tMap входная схема row1 содержит четыре String-поля, выходная — Date/Long/Long/BigDecimal:

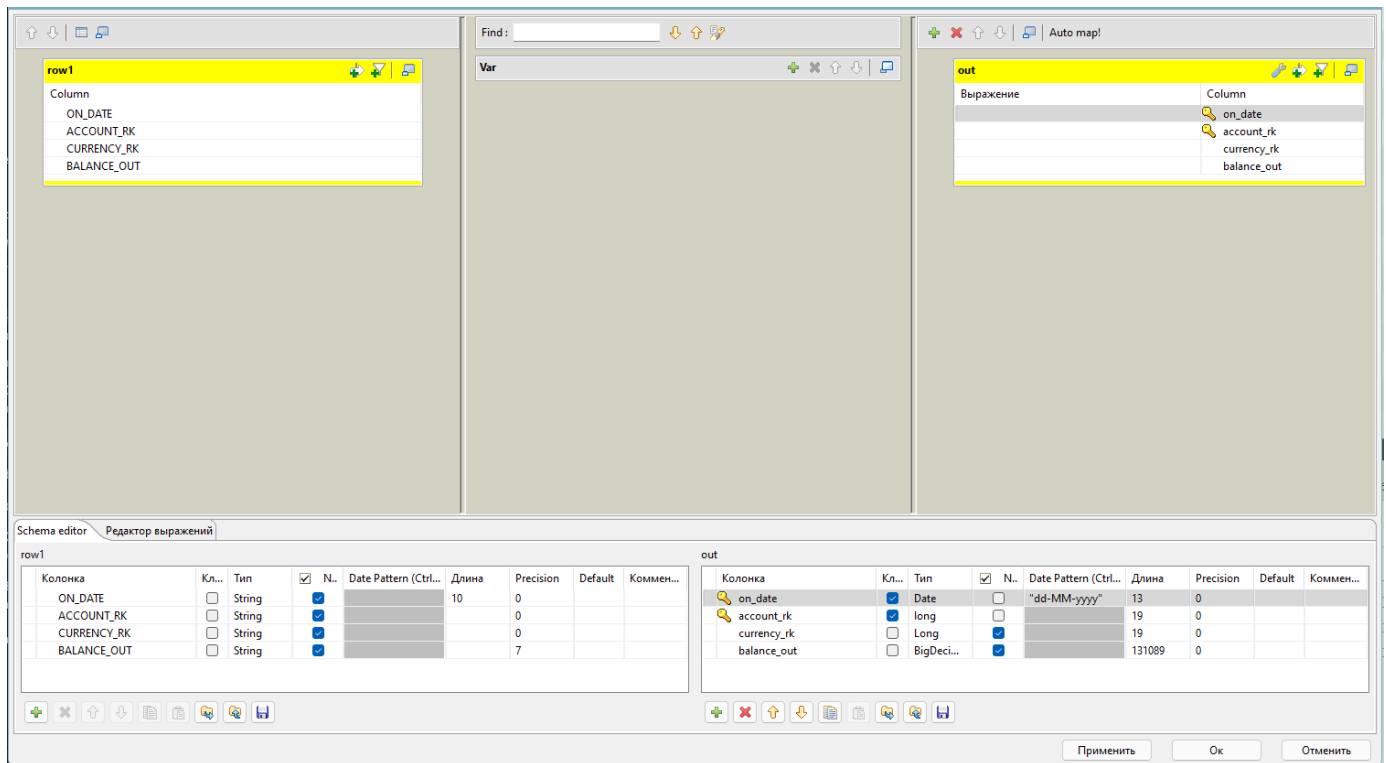


Рис. 24. Открытый *tMap* — слева *row1* (*String*), справа *out* (типизированная схема под *ds.ft_balance_f*).

Прописываем выражения преобразования:

- `on_date` — `TalendDate.parseDate("dd.MM.yyyy", row1.ON_DATE);`
- `account_rk` — `Long.parseLong(row1.ACCOUNT_RK);`
- `currency_rk` — `row1.CURRENCY_RK == null || row1.CURRENCY_RK.isEmpty() ? null : Long.parseLong(row1.CURRENCY_RK);`
- `balance_out` — `new java.math.BigDecimal(row1.BALANCE_OUT).`

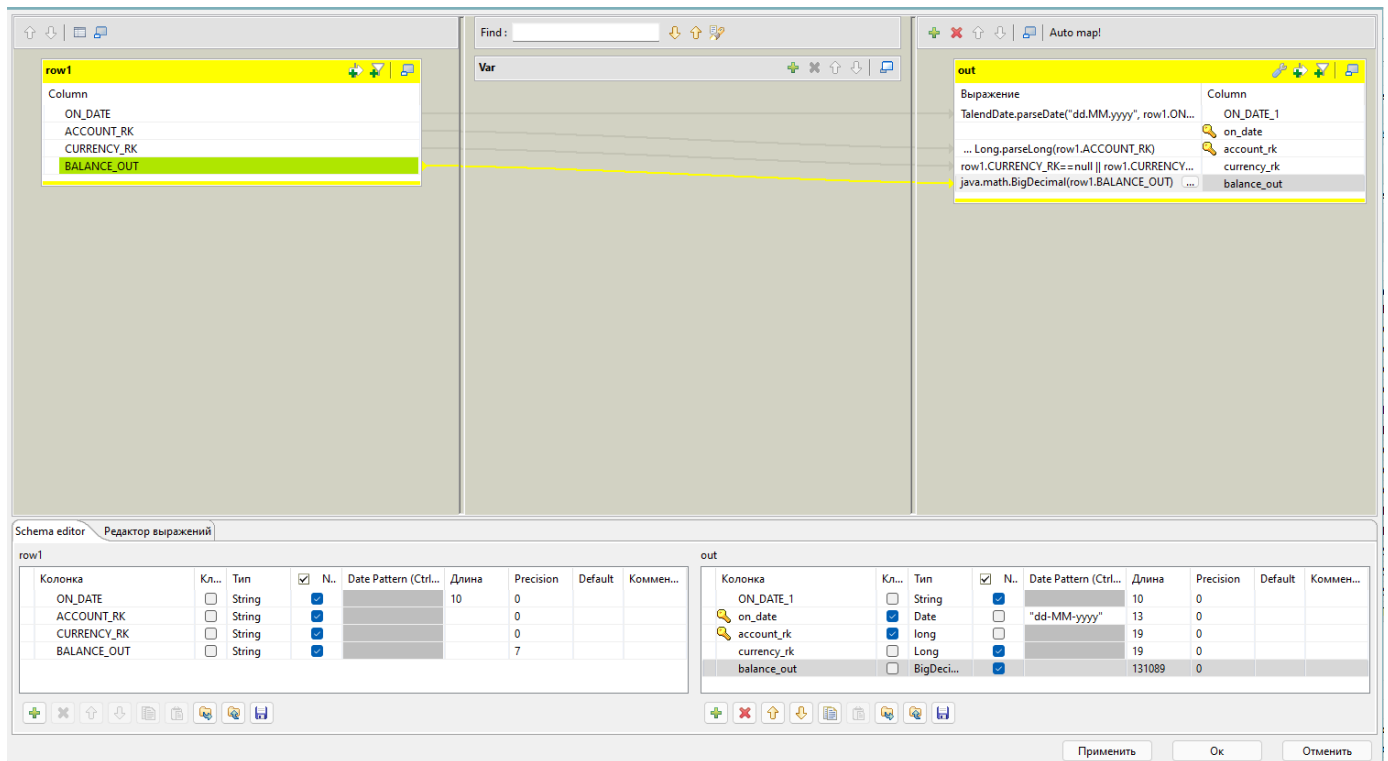


Рис. 25. Заполненные выражения tMap (на этом шаге найден лишний ON_DATE_1, потом удалён, Date Pattern исправлен на dd.MM.yyyy).

В tDBOutput выбираем готовое подключение pg_dwh, таблицу "ft_balance_f", Action on table = None (таблица уже существует), Action on data = Insert or update — это и есть запрошенный режим «Запись или замена»:

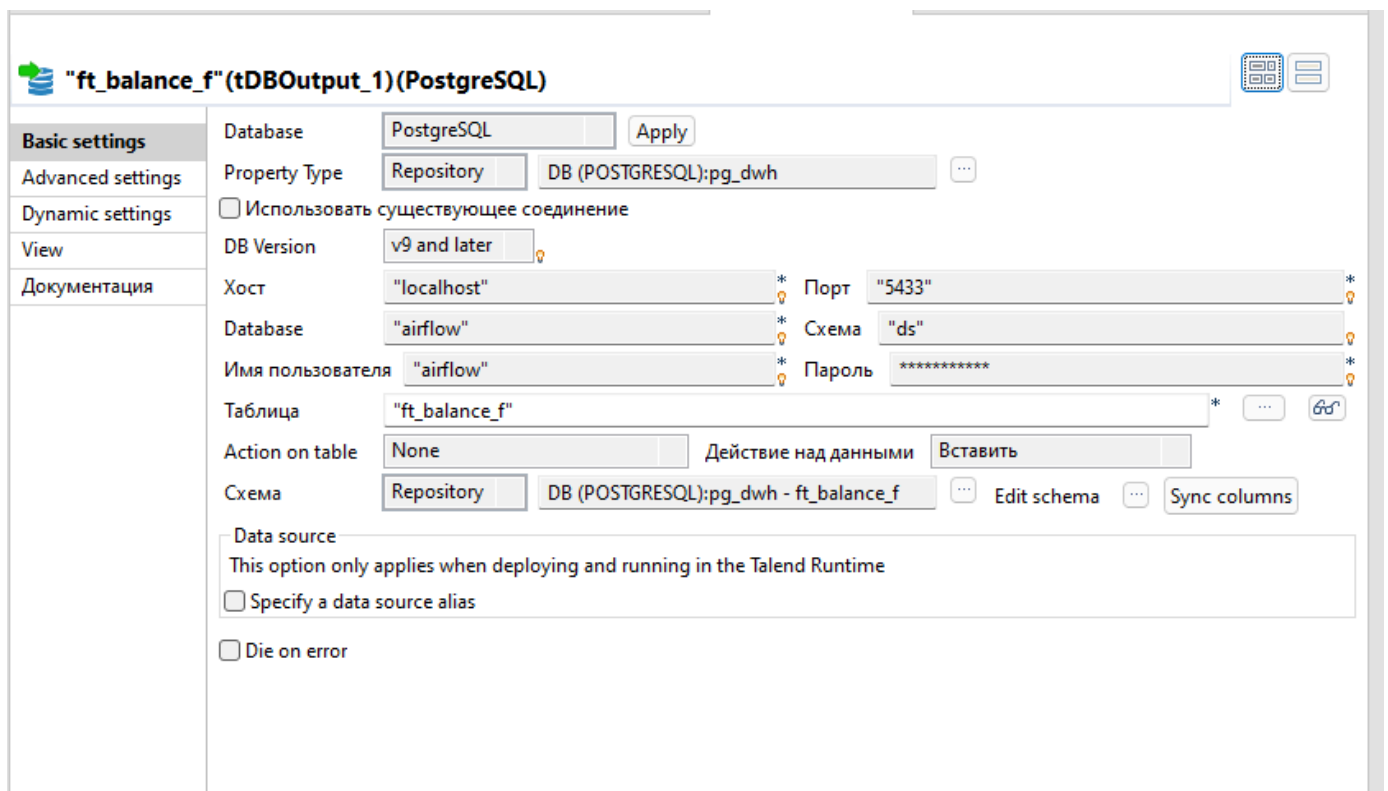


Рис. 26. tDBOutput Basic settings — pg_dwh, ds.ft_balance_f.

В Edit schema критически важно проставить флажок «Ключ» на on_date и account_rk на стороне Output — без этого Insert or update упадёт. На первом проходе ключи стояли только на Input — Talend не знал, по чему апдейтить:

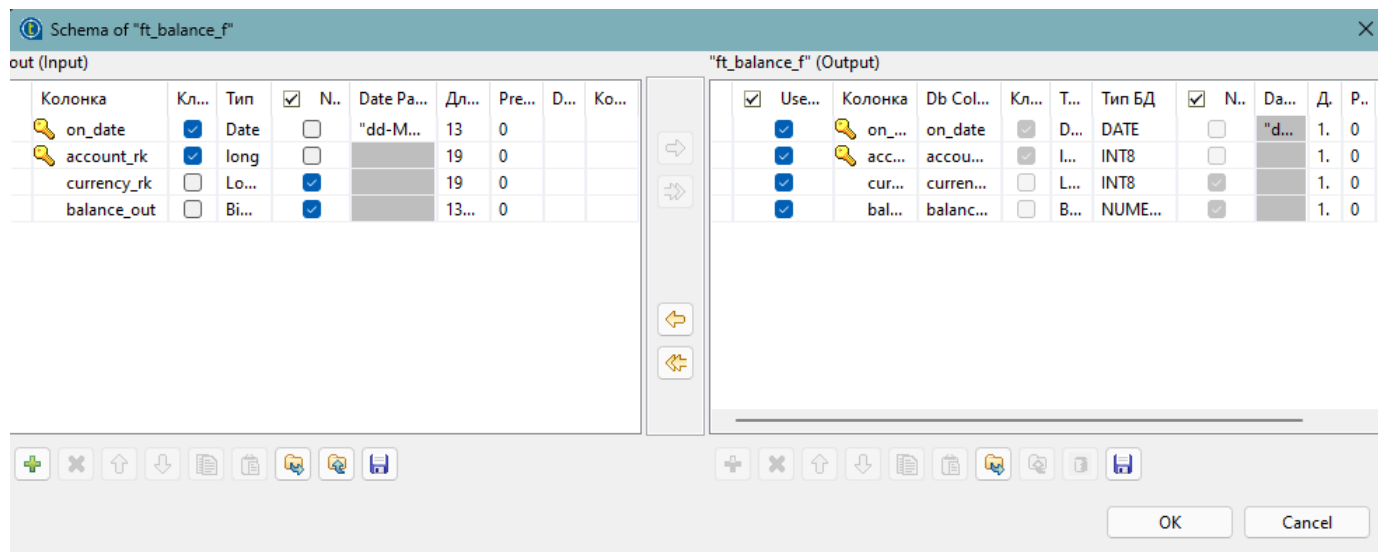


Рис. 27. Edit schema до правок — на стороне Output ключи отсутствуют.

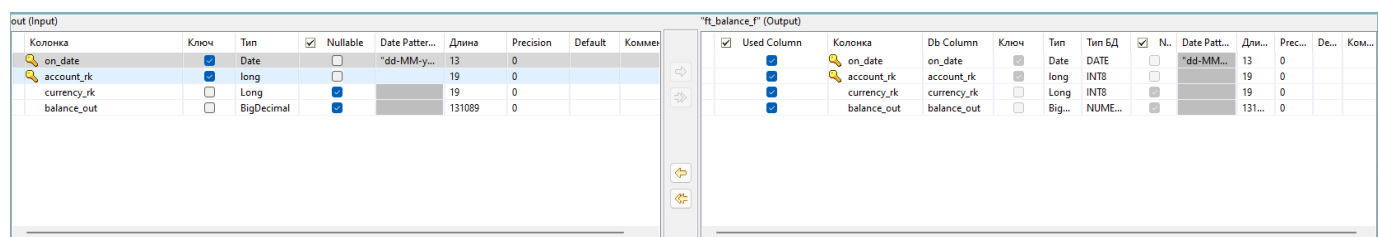


Рис. 28. Edit schema после правок — ключи on_date+account_rk на обеих сторонах, Date Pattern dd.MM.yyyy.

9.5. Запуск job в Talend

Запуск Run → Run даёт ту же картину, что и Python: 114 строк прошли через tMap и попали в ds.ft_balance_f, job завершился успешно с exit code = 0:



Рис. 29. Talend Run console: 114 rows in 1.18 s, [exit code=0] — задача 1.1 на Talend выполнена.

Полный поток (все шесть веток с tPrejob/tPostjob и таймером tSleep(5)) описан в talend/INSTRUCTION.md и собирается по той же схеме. В отчёте показан рабочий job только для ft_balance_f, чтобы продемонстрировать связку компонентов и режим Insert or update.

10. Итог

- создана отдельная схема DS, в ней шесть таблиц с правильными первичными ключами;
- создана отдельная схема LOGS, в ней универсальная таблица etl_log;
- реализован Python-ETL с COPY через TEMP-стейдж, разными масками дат и обработкой битой кодировки;
- встроен таймер на 5 секунд между событием START и фактической загрузкой;
- режим «Запись или замена» подтверждён демо на счёте 13560;
- параллельно собрана альтернативная реализация ETL-потока в Talend Open Studio для ds.ft_balance_f с режимом Insert or update — успешно отработала на тех же 114 строках.

Всё это закрывает требования задачи 1.1 курсовой работы.

11. Ссылки

Репозиторий GitHub (ветка 1_1): https://github.com/kodi842/neoflex_coursework/tree/1_1

Видео-демонстрация (Google Drive, все 4 видео по курсовой):
<https://drive.google.com/drive/folders/1ZMhXnrIE3nR8zUqTHC2uECeeShwDaIFG?usp=sharing>